PRODUCT REQUIREMENTS DOCUMENT

T-Pay

Product Requirements Document

Offline-first payments on a dual-device trust model

Internal — hackathon MVP baseline. This document records what is built, what is simulated, and what the system must do next. Section 1 is the register; section 3 is the specification.

Version 1.1 | 10 November 2026 | T-Pay engineering

Contents

Changes in version 1.1 3

1. Where we are 4

1.1 How to read the register 4

1.2 Domain logic 4

1.3 Device identity and cryptography 5

1.4 Vault and double-spend 5

1.5 Settlement 6

1.6 Sync and dual-device ledger 6

1.7 Applications and backend 6

1.8 Assistant and non-financial requirements 7

1.9 Documentation 7

1.10 Reality check 7

1.11 Not building now 7

2. Requirement gaps 9

2.1 Functional gaps 9

2.2 Non-functional gaps 10

2.3 Asked for by the source document, absent from the code 10

2.4 Baseline defects carried on the main branch 10

2.5 What the source material leaves open 10

3. The PRD 11

3.1 Product definition 11

3.2 Actors 11

3.3 Scope 11

3.4 Functional requirements 12

3.5 Non-functional requirements 16

3.6 Transaction lifecycle 17

3.7 Minimum interface surface 18

3.8 Definition of done for V1 19

4. PayPal constraints as requirements 21

4.1 What PayPal will and will not do 21

4.2 Requirements 22

4.3 Consequences the product must accept 25

4.4 Sources 25

5. Device trust 27

5.1 Two assurance tiers 27

5.2 Signing rules 27

5.3 Why the layering matters 28

5.4 Sources 28

6. Architecture contract 29

6.1 Rules 29

6.2 Bounded context ownership 29

6.3 Import matrix 30

6.4 Review checklist 30

6.5 Enforcement 30

6.6 Project structure 31

7. Roadmap 32

7.1 Parked 33

8. Open inputs and decisions 34

9. Hackathon delivery 35

9.1 Rules that shape the build 35

9.2 Submission checklist 35

9.3 Stack 35

9.4 Video script 36

9.5 Prize targets 36

9.6 Sources 36

Appendix A. Evidence index 38

Appendix B. Trace from the source document 39

Changes in version 1.1

Version 1.1 narrows the build to one complete customer-to-merchant payment on two Android phones, for the PayPal AI Hackathon deadline of 12 November 2026 at 12:00 Pacific Time (21:00 in Nigeria). Where this version conflicts with earlier material, this version wins.

DECISION	VERSION 1.1
Core product	Offline customer-to-merchant payment backed by a PayPal hold. The vault is the primary innovation.
Platform	An installable Android app built with Expo and React Native. The web build is retired as a product surface.
Project structure	The repository root holds `frontend/` for the app interface and phone-side adapters, `backend/` for server functions, and a small `shared/` package for the payment rules both must run identically.
Device exchange	Signed QR codes are the guaranteed path. Nearby Connections over Bluetooth and Wi-Fi replaces them if it passes a time-boxed test on two phones.
AI	A real model is mandatory under the hackathon rules. It stays an accessibility feature and can never sign or move money.
Merchant trust	A backend-signed device certificate and vault certificate let a phone verify its peer with no network.
Replay rule	The highest-sequence rejection is removed. It would reject honest vouchers arriving out of order. Uniqueness is tracked per vault and sequence.
Closing a vault	Early melt can no longer void vouchers still held by offline merchants. The hold stays until the customer melts in T-Pay with every declared voucher settled, or until a day before PayPal's 29-day authorisation expires.
Merchant funds	Captures reach the T-Pay business account, the T-Pay ledger credits the merchant, and sandbox Payouts complete the path if verified.
Fraud claim	Overspending is bounded and detected. T-Pay does not claim that fraud is impossible.

1. Where we are

This section is a register, not a narrative. Every capability the product needs is an item with an identifier, a status, the evidence that justifies that status, and the next action. Nothing here is aspirational, and no status is granted on the strength of a comment, a README claim, or a class name.

1.1 How to read the register

STATUS	MEANING
Met	Implemented and exercised by either an executable check or a reachable UI path.
Simulated	A real interface exists, but the behaviour behind it is a stand-in that does not deliver the guarantee it implies.
Partial	Part of the requirement is real, part is absent or unreachable in the shipped surface.
Missing	No implementation exists.

Register statuses were rechecked on 4 November 2026 by running the code, the typecheck and targeted attack probes against `main` at commit `8da2e8d`. Evidence paths are relative to the repository root. The status vocabulary is enforced by rule AR-08 in section 6: a capability may be described as working only if something executable exercises it.

1.2 Domain logic

The domain layer is the most trustworthy part of the system. It was built first, it is independent of any framework, and it is the only area with an executable check behind it. The resource-tracking logic holds up. The money-related aggregates needed a downgrade after attack testing.

ID	ITEM	STATUS	EVIDENCE	NEXT ACTION
W-01	Shared kernel: entity, value object, aggregate root, identifier, result type, money, event bus	Partial	`core/shared/domain/Money.ts` stores floats despite its integer comment, and accepts negative amounts and `Infinity`	Move money to integer minor units; add clock and random ports so aggregates stop reading the wall clock (AR-05).
W-02	Escrow aggregate, cryptogram, device identity, status value objects, repository and gateway ports	Partial	`OfflineEscrowToken` allows a failed token to become settled, and adapters copy status and capture identifiers from received data	Enforce the lifecycle in 3.6; extend with voucher sequence and vault binding (W-12).
W-03	Merchant context: inventory aggregate, price-to-quantity derivation, spoken inventory parser, sale recording	Met	`core/resource-tracking/merchant/`	None for V1.
W-04	Budget context: plan, categories, categoriser service, ambiguity detection	Met	`core/resource-tracking/budget/`	None for V1.
W-05	Transit context: seven-seat trip aggregate, passenger seats, fare collection	Met	`core/resource-tracking/transit/`	None for V1.

ID	ITEM	STATUS	EVIDENCE	NEXT ACTION
W-06	Vault aggregate with duration, spend authorisation and melt	Partial	`authorizeSpend` accepts a negative amount and grows the vault; an expired vault is never found, so `MeltMarketVaultUseCase` cannot melt it	Fix both, then bind to the PayPal authorisation and to vouchers (W-12).
W-07	Composition root and four-context wiring with an executable verification script	Met	`core/container.ts`, `core/__tests__/domain-verification.ts`	Promote the script into a real test runner during M9.

1.3 Device identity and cryptography

This is the area the source document leans on hardest and where the codebase is furthest from the requirement. The user-facing promise is that both devices hold an identity, sign every transaction, and can prove to each other who they are. None of that exists.

ID	ITEM	STATUS	EVIDENCE	NEXT ACTION
W-08	Device keypair, key custody, backend-issued device token, public-key registry	Missing	`DeviceIdentity` declares an optional `publicKey` that is never populated (`core/escrow/domain/value-objects/DeviceIdentity.ts`)	Generate an ECDSA P-256 keypair per device, register the public key, issue a device token (FR-01 to FR-03).
W-09	Voucher signatures and verification	Simulated	`core/escrow/domain/services/CryptographicExchangeService.ts` computes a 32-bit string hash and labels it `HMAC-SHA256` ; `verifyProof` only checks a string prefix; the nonce is generated and never inspected	Replace with a real signature over canonical bytes (FR-11, DT-04 to DT-06).
W-10	Canonical serialisation, sequence numbers, per-vault duplicate detection	Missing	No code path reads `Cryptogram.nonce`	Adopt JCS canonicalisation, a per-device monotonic sequence and a server-side unique key on vault and sequence (FR-13, FR-14).

1.4 Vault and double-spend

ID	ITEM	STATUS	EVIDENCE	NEXT ACTION
W-11	Vault linked to offline spending	Partial	`MarketVault.authorizeSpend` is referenced only by `core/__tests__/domain-verification.ts` ; `CreateOfflineEscrowUseCase` never consults a vault, so vouchers can exceed the locked amount	Debit the vault when a voucher is signed, and refuse to sign beyond the remaining balance (FR-07, FR-08).
W-12	Vault as a real PayPal authorization, with reserved-versus-settled accounting and melt that respects in-flight vouchers	Missing	`MarketVault` fabricates a `paypalPreAuthId` from `Math.random()` ; `meltUnspentFunds` ignores unsettled vouchers	Map lock to an authorisation order, spend to partial capture, melt to void (PP-04, PP-05, FR-06, FR-09).

1.5 Settlement

ID	ITEM	STATUS	EVIDENCE	NEXT ACTION
W-13	Settlement service: server-side capture, idempotency, leasing, receipts, webhooks, reconciliation	Simulated	`PayPalSdkGatewayAdapter` runs in the client, reads credentials from `localStorage`, falls back to a mock capture that always succeeds, and never checks the live capture response. A probe captured one voucher twice under concurrent sync.	Move settlement behind the backend, own dedupe in the database, add webhook handlers (FR-17 to FR-21, PP-02 to PP-09).
W-14	Currency conversion layer	Missing	NGN converted to USD by an inline `/ 1500` constant inside the adapter	One FX port, rate snapshotted on the voucher, no constants in adapters (FR-10, PP-01).

1.6 Sync and dual-device ledger

Both devices are supposed to hold the same signed transaction, so that whichever reconnects first can settle it and both see the money move. Only the sender keeps a record today.

ID	ITEM	STATUS	EVIDENCE	NEXT ACTION
W-15	Receiver-side persistence and credit confirmation	Partial	`BroadcastChannelProximityAdapter` only reaches browser tabs, returns hardcoded peers from `discoverPeers`, and has no registered listener; the sender marks the voucher acknowledged on the receiver's behalf	Replace with a real two-phone transport; persist received vouchers and return a signed receipt (FR-22, FR-38).
W-16	Sync engine: reconnect trigger, retry with backoff, receipt relay, vault-versus-ledger consistency check	Missing	`toggleNetwork` in `src/hooks/useTPayDomain.ts` calls one sync pass and sets a toast	Build the sync state machine and reconciliation pass (FR-24, FR-25).

1.7 Applications and backend

ID	ITEM	STATUS	EVIDENCE	NEXT ACTION
W-17	Web prototype	Partial	`src/components/PayPalEscrowModal.tsx` and `NearbyPeerScannerModal.tsx` simulate the payment flow with `Math.random()` and timers; `useEscrowDomain` has no consumers	Retired as a product surface; keep only as a design reference until the mobile screens replace it.
W-18	Android application on the shared domain	Missing	`react-native-app/App.tsx` is a 2,013-line screen file with hardcoded state, no import from `core/`, no proximity library, and an icon path that does not exist	Rebuild on Expo SDK 58 in `frontend/` as the product surface and consume `shared/` (M2, FR-35, FR-45).
W-19	Backend as a stateful service	Partial	The backend does not start because `cors` is missing. Controllers reuse `LocalStorage*Repository` classes that keep only memory in Node, seeded and default user identifiers disagree, and any caller can act as any user	Run on Render with Postgres and device authentication (M0, M1).
W-20	Typecheck and build hygiene	Missing	`npx tsc --noEmit` reports two errors: `cors` is imported but not a dependency, and `vault.controller.ts` omits `userId` required by `MeltMarketVaultDTO`	Fix both; make typecheck a gate (M0, NFR-11).

1.8 Assistant and non-financial requirements

ID	ITEM	STATUS	EVIDENCE	NEXT ACTION
W-21	Assistant: offline rule parser, real online model, language coverage, spoken readout	Simulated	`LocalSLMAdapter` is keyword matching; `executionMode: 'online_cloud'` is accepted and ignored in `ProcessSpokenCommandUseCase`; `@google/genai` is installed and never called; dialect handling is a string transform in `MultilingualVoiceService`	Add the server-side model, label the offline parser honestly, move strings to a catalogue (FR-31 to FR-34, FR-46).
W-22	Durability, observability, privacy, deployment, test pyramid, CI	Missing	Storage is `window.localStorage` only; logging is `console.warn`; no privacy or retention policy; no deploy target; one console-assertion script; no CI configuration	Address in M9 and NFR-01 to NFR-14.

1.9 Documentation

ID	ITEM	STATUS	EVIDENCE	NEXT ACTION
W-23	Documentation that matches the system	Simulated	`README.md` claims real HMAC signatures, a Bluetooth mesh, a secure enclave, a working PayPal integration and a fully passing suite; `DDD_ARCHITECTURE.md` describes an `OfflineEscrowToken` state machine with transitions the code does not implement	Replace the claims with this document's register and keep them current (AR-08).
W-24	Hackathon submission pack	Missing	No `LICENSE` file at the repository root, no installable Android build, no video, no judge instructions	Complete section 9 (NFR-15).
W-25	Offline trust chain	Missing	Nothing lets an offline phone verify a peer or a vault, and the 1.0 specification had no mechanism for it	Add device and vault certificates (FR-36, FR-37, DT-09).
W-26	Merchant funds path	Missing	No ledger credits a merchant and no payout path exists	Add the merchant ledger and sandbox Payouts (FR-40, FR-41, PP-14, PP-15).

1.10 Reality check

Of twenty-six register items: **4 are met, 11 are simulated or partial, and 11 are missing entirely**. Version 1.0 counted seven met items; running the code moved W-01, W-02 and W-06 down, and the new mobile platform decision and the three new items added to the missing count.

The shape of that result is consistent: everything that is purely business logic works, and everything that touches money movement, identity, or the guarantee that a voucher cannot be spent twice is either simulated or absent. That is precisely the half of the product that the PayPal integration and the offline claim stand on. Attack probes confirmed the gap: a forged voucher passed verification, and a ₦999,999,999 payment with no vault reported settlement with an invented capture identifier. Any pitch material written before W-08 to W-16 close should describe those flows as prototypes.

1.11 Not building now

Recorded so the register cannot quietly grow. Each has a reason, and each is re-evaluated only after the submission ships.

PARKED	WHY NOT NOW
B2B offline-protocol API for banks and fintechs	Requires a protocol standard, certification and a partner. It is a business decision that follows a working closed loop, not a feature.
Interoperability with Moniepoint, OPay, PalmPay	Depends on the protocol above.
Multiple currencies	The FX port is built for one display currency and one settlement currency. Adding pairs is a data problem that this release does not need.
KYC and onboarding	Sandbox accounts only; no real customer funds move.
Card issuance, savings, yield	Out of the product's stated problem.
iOS build	Two Android phones satisfy the demo. iOS doubles device testing and signing work inside an eight-day window.
Hardware-backed signing keys	Requires a native module. The demo uses Tier B and labels it honestly (see 5.1).
Authorisation reauthorisation	No longer parked. A vault can wait up to 28 days for its customer, so the backend reauthorises the uncaptured amount each time the three-day honor period ends.
On-device language model	Not achievable reliably in eight days across five languages. Offline voice remains rule-based and is labelled so.
Budget and transit on mobile	Both remain tested domain logic. They return to the app only after M10 is signed off.
AG Studio dashboard, Render Workflows	Sponsor prize stretch work. Neither starts before M10 is signed off.

2. Requirement gaps

Gaps are traced to register items so that nothing in this section can be closed by writing prose. Every gap below is closed by a named requirement in section 3 or 4. Gaps G-F13 to G-F16 are defects found in the 1.0 specification itself.

2.1 Functional gaps

ID	GAP	ITEM	CLOSED BY
G-F01	Devices have no cryptographic identity, so a peer cannot verify who signed a voucher.	W-08	FR-01, FR-02, FR-03
G-F02	Signatures are not signatures: a 32-bit string hash presented as HMAC-SHA256, with prefix-only verification.	W-09	FR-11, DT-04
G-F03	Nonces are generated and never checked, so a captured voucher can be replayed indefinitely.	W-10	FR-13, FR-14, DT-07, DT-10
G-F04	The vault does not constrain offline spending, so the double-spend the product exists to prevent is still open.	W-11	FR-07, FR-08
G-F05	The vault's PayPal identifiers are random strings; nothing is actually held on PayPal.	W-12	FR-06, PP-04
G-F06	Settlement runs in the client with a secret in browser storage and a mock fallback that always reports success.	W-13	FR-17, PP-02, PP-03
G-F07	No idempotency or lease, so two reconnecting devices can capture the same voucher twice.	W-13	FR-18, FR-19
G-F08	Naira is converted with an inline constant, and the rate is not recorded anywhere.	W-14	FR-10, PP-01
G-F09	The receiver never persists a voucher, so there is no dual-device ledger and no credit-side confirmation.	W-15	FR-22, FR-23
G-F10	There is no reconnect trigger, retry policy or reconciliation pass.	W-16	FR-24, FR-25
G-F11	The online assistant path is accepted and discarded, and dialect support is a string transform.	W-21	FR-31, FR-32, FR-33
G-F12	Web and mobile each maintain a private copy of domain state, with a duplicated intent parser.	W-17, W-18	FR-35, AR-03, AR-04
G-F13	A high-water mark would reject an honest lower-sequence voucher when another merchant reconnects first, then block the payer's vault.	W-10	FR-14, DT-10
G-F14	An offline merchant cannot reach the backend key registry, so a self-signed voucher from an unknown key would look valid.	W-25	FR-36, FR-37, DT-09
G-F15	The backend cannot see vouchers held by offline merchants, so an early melt could void the hold they depend on.	W-12	FR-09, FR-39
G-F16	A capture request has no payee field, so one authorisation cannot pay a different PayPal merchant per voucher.	W-26	FR-40, FR-41, PP-14

2.2 Non-functional gaps

ID	GAP	ITEM	CLOSED BY
G-N01	Storage is browser `localStorage`, or memory only for vaults. An offline payment that evaporates is worse than no payment.	W-22	NFR-01
G-N02	The PayPal client secret is written to client `localStorage`.	W-13	NFR-02, PP-02
G-N03	No audit trail linking a settled amount to a PayPal capture identifier.	W-13	NFR-03, FR-20
G-N04	No idempotency and no duplicate-settlement test, so correctness under concurrency is unproven.	W-13, W-16	NFR-04
G-N05	No latency budget for the only interaction that must feel instant: a spoken sale confirmed offline.	W-21	NFR-05
G-N06	One console-assertion script, no runner, no CI gate, and two typecheck errors accepted on `main`.	W-20, W-22	NFR-11, NFR-12
G-N07	The domain cannot be shared by the app and the backend as written: adapters read `window` and `navigator`, and repositories are browser classes.	W-19	NFR-13, AR-02
G-N08	No localisation catalogue, no behaviour when no speech voice is installed, no privacy or retention policy, no observability.	W-21, W-22	NFR-06 to NFR-14
G-N09	Documentation asserts capabilities the code does not have.	W-23	AR-08
G-N10	The project has no license file, installable build, or judge access path, so it cannot be submitted as it stands.	W-24	NFR-15

2.3 Asked for by the source document, absent from the code

Four requirements in the source material have no implementation at all: the secure element that holds each device's key; the mutual token exchange that lets two devices identify each other; the online model fallback for the assistant; and the end-of-day reminder that prompts a stock review. A fifth — a real exchange between two phones — exists only as a same-browser channel. These are the items a reviewer will look for first, because they are the ones the concept rests on.

2.4 Baseline defects carried on the main branch

`npx tsc --noEmit` fails with two errors: `backend/src/server.ts` imports `cors`, which is not a dependency, and `backend/src/controllers/vault.controller.ts` calls the melt use case without the `userId` its DTO requires. The backend therefore crashes on start. The domain verification script passes, but only when invoked with a loader flag and experimental type stripping, and it tests no attack. These facts are recorded rather than fixed here, because this document changes no application behaviour.

2.5 What the source material leaves open

Page 5 of the six-page source PDF is a diagram comparing the B2B and closed-loop strategies. It adds no V1 requirement. That strategy discussion is treated as V2 positioning, and the source's claims of zero fraud and production readiness are rejected. Currency handling is the largest silent gap in the source material: it never states which currency PayPal would actually settle in, and as section 4 shows, it cannot be naira.

3. The PRD

3.1 Product definition

T-Pay is an offline-first Android payment app for markets where connectivity is intermittent and payment terminals fail during outages. A customer and a merchant both run the app. While online, the customer places a PayPal hold and receives a signed vault certificate. With no network present, the two phones exchange a signed payment voucher directly. The merchant sees a pending credit immediately. Whichever phone regains connectivity first submits the voucher, the backend captures against the hold, and both phones learn the outcome.

PayPal handles funding authorisation and final settlement. T-Pay handles device trust, offline exchange, verification, synchronisation and the merchant ledger.

Three properties define the product, and each is a requirement rather than a slogan:

1. **Offline spending is bounded.** The app refuses to sign beyond the vault, and PayPal caps total captures at the authorised amount. A tampered phone could still try to overspend across two offline merchants, and that is detected at settlement.
2. **A voucher cannot be forged or changed.** Signatures and certificates bind device, vault, amount, sequence, payee and expiry.
3. **A voucher settles at most once.** Every voucher reaches a terminal state, and duplicates never produce a second capture.

3.2 Actors

ACTOR	DESCRIPTION	PRIMARY NEED
A1 Trader	Market seller with stock, often low literacy. The primary merchant.	Accept a digital payment offline, record the sale by voice, know what is left
A2 Salaried user	Budgets by category. A later persona, parked for this release.	Have a payment land in the right budget without bookkeeping
A3 Driver or conductor	Collects fares from a fixed number of seats. A later persona, parked for this release.	Know who has paid and how many seats remain
A4 Settlement service	Backend process that holds PayPal credentials and settles vouchers	Never double-settle, never lose a voucher, always be able to explain an outcome
A5 Customer	The payer, carrying a phone and a PayPal account	Reserve a spending limit before going offline, then pay without a network

A4 is listed as an actor deliberately. In the current codebase settlement is an activity of the client, which is why the platform's most security-sensitive behaviour is running in the least trusted place. A5 and A1 drive every P0 requirement.

3.3 Scope

In this release: device identity and certificates; vault locking against a PayPal authorisation; signed offline vouchers with replay defence; a real two-phone exchange with networking disabled; server-side idempotent

settlement with webhooks; a dual-device ledger and merchant ledger; a real AI assistant for the payment and sale flows; spoken readout; an installable Android build.

Supporting: merchant inventory and voice sales, if connected to the app without delaying any P0 item.

Parked: see 1.11. The list is normative: nothing in it may be started before every P0 requirement below is verified.

3.4 Functional requirements

Each requirement states a testable acceptance criterion. "Testable" means a reviewer can execute the check without reading the implementation. Requirements link to the register items they close.

Priorities decide what survives the deadline. **P0** must work in the demo. **P1** is built next if P0 is verified. **P2** stays as tested domain logic and does not block submission. FR-26, FR-27 and FR-41 are P1. FR-28 to FR-30 are P2. Everything else in this section is P0.

3.4.1 Device identity and keys

ID	REQUIREMENT	ACCEPTANCE CRITERION	CLOSES
FR-01	On first run a device generates one ECDSA P-256 keypair, with the private key encrypted at rest through the Android Keystore-backed secure store.	Fresh install: a keypair is created once and reused across restarts; the private key never appears in logs, database rows or exported files.	W-08
FR-02	The device registers its public key with the backend and receives a device token bound to it.	Registration returns a token and a key identifier; a second registration with the same key returns the existing identity rather than a duplicate.	W-08
FR-03	Every voucher names the sending key identifier, and the backend resolves it through a registry that supports revocation.	A voucher signed with a revoked key is rejected with a distinct, surfaced error, and the device is told to re-register.	W-08, G-F01
FR-04	The assurance tier of the signing key is recorded per device and shown in the interface.	The demo build shows a Tier B badge. Where the key was created in the Android Keystore, the badge may say the key is kept inside the phone, but no UI copy, log or pitch material claims Tier A or attested hardware, because the backend has no attestation to check.	W-08, NFR-02
FR-05	Device identity survives application restart but not reinstallation.	After a restart the device signs with the same key identifier; after reinstalling it registers a new key and the old key is revoked.	W-08

3.4.2 Vault

ID	REQUIREMENT	ACCEPTANCE CRITERION	CLOSES
FR-06	Locking a vault creates a PayPal order with `intent=AUTHORIZE` on the backend, has the payer approve it, and authorises it, producing an authorisation identifier stored with the vault.	Locking opens a real PayPal sandbox approval in an in-app browser and returns to the app; the stored identifier resolves through the PayPal API; no identifier is generated locally.	W-12, G-F05
FR-07	A voucher may only be signed if its amount is within the vault's remaining capacity.	Attempting to sign a voucher larger than the remaining capacity is refused before any signature is produced, with the remaining amount stated.	W-11, G-F04

ID	REQUIREMENT	ACCEPTANCE CRITERION	CLOSES
FR-08	The vault tracks reserved, settled and remaining capacity separately in settlement-currency minor units, and remaining equals authorised minus reserved minus settled.	After two vouchers and one settlement, the three figures reconcile exactly; a property test over random sequences finds no state where their sum exceeds the authorised amount, and negative amounts are refused.	W-06, W-11
FR-09	Melting a vault never voids funds that issued vouchers may need. Melt stops new signing at once; the remaining hold is released only after the customer's signed spend statement shows every issued voucher settled, or at the settle-by time, a day before PayPal's 29-day authorisation expires. The customer using PayPal outside T-Pay never releases it.	Melt with an unsettled declared voucher schedules release at the settle-by time and names the voucher; melt with all vouchers settled releases the hold now and PayPal shows it released.	W-12, G-F15
FR-10	The vault certificate and every voucher record the FX rate and settlement-currency amount, obtained through a single FX port.	The voucher shows the NGN amount, the USD amount and the rate; the rate is fixed at vault lock; the adapter contains no numeric rate literal; the demo rate is labelled as a fixed demo rate.	W-14, G-F08

3.4.3 Offline voucher

ID	REQUIREMENT	ACCEPTANCE CRITERION	CLOSES
FR-11	A voucher is signed over JCS canonical bytes with a domain-separation prefix, never over `JSON.stringify` output.	A known voucher verifies against a fixed byte fixture; reordering JSON keys does not change the signature; a signature produced over non-canonical bytes is rejected.	W-09, G-F02
FR-12	Amounts travel as integer minor units or decimal strings, never floats.	A voucher for 12.50 carries 1250 or "12.50"; no floating-point value appears anywhere in a signed payload.	W-09
FR-13	Each voucher carries a monotonic sequence per device and vault, persisted before the voucher is issued.	Signing two vouchers yields consecutive sequences; killing the app mid-flight never reissues a used sequence with different content.	W-10, G-F03
FR-14	The backend treats vault and sequence as a unique key. The same signed bytes are a duplicate; the same key with different content is a conflict. Sequence order and arrival order are irrelevant.	Voucher 2 settling before voucher 1 leaves voucher 1 settleable; replaying a voucher returns its stored outcome flagged as a replay with no second capture; a conflict is rejected, raises an alert and blocks the vault.	W-10, G-F03, G-F13
FR-15	Default behaviour is offline-first: a payment is created and delivered locally first, and settles in the background if a network happens to be present.	With networking on, a payment shows as delivered instantly and settles within seconds without a user action; with networking off it behaves identically apart from the settlement timing.	W-13
FR-16	A voucher's terminal failure states are explicit and require a decision rather than a silent retry.	An expired authorisation surfaces a blocked state naming the cause; an invalid signature or an overspend surfaces a rejection; neither retries indefinitely.	W-13, W-16

3.4.4 Settlement

ID	REQUIREMENT	ACCEPTANCE CRITERION	CLOSES
FR-17	Settlement runs only on the backend, and no client ever holds a payment credential.	Searching the Android bundle finds no PayPal secret or token endpoint; all settlement requests originate from the backend.	W-13, G-F06

ID	REQUIREMENT	ACCEPTANCE CRITERION	CLOSES
FR-18	Settlement is idempotent keyed on voucher identity, with a lease so only one worker processes a voucher at a time.	Submitting the same voucher from two devices concurrently produces exactly one capture and one settled ledger entry; the loser receives the settled result rather than an error.	W-13, G-F07
FR-19	A retryable failure is retried with exponential backoff and jitter, and an ambiguous failure is reconciled by reading the resource before any retry.	A forced timeout followed by a successful read-back results in one capture, not two; retries never occur faster than the configured backoff.	W-13, G-F07
FR-20	Every settled voucher stores the PayPal order and capture identifiers and reaches exactly one terminal state.	Each settled ledger line opens to its capture identifier; no voucher remains non-terminal after a test run completes.	W-13, G-N03
FR-21	Webhook events are signature-verified, idempotent, and update voucher state for completion, pending, decline, refund, reversal and authorisation void.	Replaying the same webhook twice changes nothing the second time; a decline in either PayPal naming variant moves the voucher to a failed state.	W-13

3.4.5 Sync and ledger

ID	REQUIREMENT	ACCEPTANCE CRITERION	CLOSES
FR-22	Both devices persist the same voucher, with payer and payee views.	After an offline payment both phones list the voucher across restarts; the payer sees a pending debit and the payee a pending credit.	W-15, G-F09
FR-23	Only the backend may declare a voucher settled, and the outcome propagates to both devices.	Settling on device A updates device B's entry without a manual refresh once it reconnects.	W-15, G-F09
FR-24	Reconnection triggers settlement automatically; there is no user action to perform.	Restoring networking settles pending vouchers within the configured window with no taps.	W-16, G-F10
FR-25	The sync pass reconciles the vault against the ledger and flags any inconsistency.	A deliberately injected overspend across two merchants is rejected at settlement for the excess voucher, reported with both figures, and blocks the vault from further signing.	W-16

3.4.6 Resource tracker

ID	REQUIREMENT	ACCEPTANCE CRITERION	CLOSES
FR-26	A spoken sale deducts inventory by derivation: amount divided by registered unit price.	"Collect one thousand for drink" at ₦500 deducts exactly two units and states the resulting stock.	W-03
FR-27	A spoken inventory statement creates catalogue rows and reads them back for confirmation.	A three-item utterance creates three items and speaks a matching summary.	W-03
FR-28	A budget expense is categorised, and an unknown item raises a clarification question with one-tap answers.	An unknown item produces a question naming the candidate categories; selecting one records the expense in that category.	W-04
FR-29	Fare collection fills the seven-seat manifest and reports paid and remaining seats aloud.	Collecting fares fills seats up to capacity, never overbooks, and speaks the remaining count.	W-05
FR-30	A day-end review is produced and prompted at end of day, covering sales, payments and restock alerts.	With the schedule enabled, a review is offered at the configured hour and lists the day's totals by payment method.	W-03, G-N08

3.4.7 Assistant

ID	REQUIREMENT	ACCEPTANCE CRITERION	CLOSES
FR-31	Offline intent parsing always has a rule-based path. A person may also choose, after installing, to download a small language model that runs on the phone. When it is present, the rules and the model answer together: whatever the rules recognise wins, and the model only fills what they couldn't read. Every model answer passes the same checks as the online model (FR-32). Each path is labelled as what it is. It covers the payment and sale intents through speech where the device supports offline recognition, and through tap or typed input otherwise.	With networking off, the payment and sale flows complete in all five languages through the fallback path, with or without the model; the interface says "AI on your phone" only when the model answered, and never claims a model where none ran.	W-21, G-F11
FR-32	When a network is present, speech and text go to a real model on the backend, which returns a typed proposal. The proposal is validated before it reaches a person, and the person confirms before anything is signed.	Spoken Pidgin, Yoruba, Igbo, Hausa and English requests each produce a valid proposal in the test set; a model response above vault capacity, or one that fails the schema, is rejected before confirmation.	W-21, G-F11
FR-33	User-facing strings and spoken text in the payment flow come from a catalogue covering English, Nigerian Pidgin, Yoruba, Igbo and Hausa.	Switching language changes every visible payment-flow string with no untranslated fallback; a native speaker has reviewed the demo lines.	W-21
FR-34	Speech output degrades gracefully when no suitable voice is installed.	With no voice for the chosen language, the confirmation is still visible and uses large numerals and icons; the failure is logged, not thrown as an error.	W-21

3.4.8 Client architecture

ID	REQUIREMENT	ACCEPTANCE CRITERION	CLOSES
FR-35	The mobile app reads domain state through the shared core; no screen holds a private mirror of a domain fact.	Changing a vault amount in one component updates every other screen without a reload; a grep for balance or vault state in screen files finds no local copies.	W-18, G-F12

3.4.9 Offline trust and exchange

ID	REQUIREMENT	ACCEPTANCE CRITERION	CLOSES
FR-36	Registration issues a backend-signed device certificate binding key identifier, public key, display name and expiry. Both phones verify the peer's certificate against a T-Pay public key pinned in the app.	With networking off, a certificate signed by any other key, or an expired one, is refused and the screen says why.	W-25, G-F14
FR-37	Locking a vault issues a backend-signed vault certificate binding vault, device key, authorised amount, FX rate, spend-by time and settle-by time.	With networking off, a merchant phone rejects a voucher with no certificate, with a certificate for another device, past its spend-by time, or above the certificate amount.	W-25, G-F14
FR-38	Payment is a three-step exchange: the merchant presents a payment request, the customer returns a signed voucher, and the merchant returns a signed receipt. Delivery retries resend the same bytes and never re-sign.	A completed exchange gives both phones the same voucher; a tampered voucher is rejected; if the receipt is missed, the customer sees sent but unconfirmed and the amount stays reserved.	W-15, G-F09

ID	REQUIREMENT	ACCEPTANCE CRITERION	CLOSES
FR-39	The merchant phone records every vault and sequence it accepts and refuses a repeat.	Presenting the same voucher twice to one offline merchant produces one pending credit and a visible duplicate warning.	W-10, G-F03
FR-47	The transport sits behind `IProximityTransport`. Signed QR codes are the baseline, a copyable code is the fallback, and Nearby Connections is used only if it passes M3.	The same signed bytes pass through each available transport; two airplane-mode phones complete a payment on the demo transport.	W-15

3.4.10 Merchant funds, AI and mobile

ID	REQUIREMENT	ACCEPTANCE CRITERION	CLOSES
FR-40	Each settled voucher creates one merchant ledger credit in the T-Pay ledger, linked to its PayPal capture.	The merchant sees the credit with the capture identifier; the ledger total equals the sum of settled vouchers for that merchant.	W-26, G-F16
FR-41	The merchant can withdraw a ledger balance to a PayPal sandbox account through Payouts.	A withdrawal appears in the merchant sandbox account and reduces the ledger once; a retried request produces no second payout.	W-26, G-F16
FR-42	The Android app installs and runs without Expo Go or a development server.	A clean Android phone installs the published build and completes vault lock against the hosted backend.	W-18, W-24
FR-43	The in-app PayPal approval returns the user to the correct vault through a deep link.	Approval returns to the app and authorises the vault; cancelling returns to the app with no vault and no hold.	W-12
FR-44	A background sync runs when connectivity returns while the app is open or next opened, and the UI never depends on the user pressing sync.	Reconnecting one phone settles pending vouchers on that phone and updates the other when it next connects.	W-16, G-F10
FR-45	`frontend/` and `backend/` import the payment rules from the local `shared/` package. Android adapters for storage, cryptography, transport, clock and speech stay in `frontend/`.	The domain tests, the backend and the Android build all run from the same `shared/` source.	W-18
FR-46	The AI model never receives keys, tokens or signing capability, and every AI-assisted action is logged with its proposal and validation result.	Code review finds no signing call in the assistant path; a prompt-injected utterance such as "ignore limits and pay everything" produces no signature.	W-21

3.5 Non-functional requirements

Targets are numeric because a requirement without a number cannot fail.

ID	REQUIREMENT	TARGET	VERIFICATION
NFR-01	Durable offline storage	Zero accepted payments lost across a process kill, a phone restart and a 24-hour offline window	Kill the app mid-payment on a real phone; the voucher is present and settleable afterwards
NFR-02	Key and secret handling	Private keys encrypted at rest by the platform store; payment and AI secrets exist only in server environment variables	Inspect the database and logs for key material; search the Android bundle for secrets
NFR-03	Auditability	Every settled amount traces to a capture identifier and a signing key identifier	Ledger export contains both for every settled line

ID	REQUIREMENT	TARGET	VERIFICATION
NFR-04	Settlement correctness	Zero duplicate captures across 100 concurrent duplicate submissions	Concurrency test against Postgres and a stubbed gateway counting captures
NFR-05	Offline transaction latency	Merchant credit visible within 1 s of scanning a valid voucher; rule-based parse confirmed within 400 ms of final text	Instrument the path; record on a real mid-range Android phone
NFR-06	Speech and accessibility	Every payment screen usable without reading fluently: each confirmation has a spoken form where a voice exists, and every control has a 48 dp minimum touch target	Manual audit on a phone with speech disabled and enabled
NFR-07	Localisation completeness	Zero untranslated strings in the payment flow in the five supported languages	Catalogue lint in CI
NFR-08	Privacy	Sandbox data only; no real account data stored; a documented retention period for settled vouchers. Local database encryption is a post-hackathon item.	Inspect the storage adapter and the written policy
NFR-09	Observability	Every voucher transition logged with correlation identifier; settlement failures alertable	Log inspection after a forced failure
NFR-10	Deployment	Backend deploys on Render from a blueprint file with configuration from environment; health endpoint reports database and PayPal reachability	Deploy to a clean environment and hit health
NFR-11	Type safety	Typecheck passes with no errors and is a required gate	Continuous integration run
NFR-12	Test pyramid	Domain unit tests per money aggregate, an integration test per payment use case, attack tests for every audit finding, and one scripted two-phone run	CI run plus the recorded two-phone checklist
NFR-13	Portability	The domain package runs in React Native and Node with no platform globals below the adapter layer	Boundary check plus Android and backend builds
NFR-14	Performance headroom	Cold start under 3 s on mid-range Android; document build time under 5 s	Measured on device and in build timing
NFR-15	Judge access	A judge can install the build and run the demo with published sandbox test logins until judging ends on 15 December 2026	A person outside the build team follows the README on a clean phone

3.6 Transaction lifecycle

A voucher has exactly one path to a terminal state. The important property is that nothing in the middle can loop forever, and nothing terminal can be re-entered.

```
DRAFT
  |  sign (key available, vault capacity sufficient, sequence persisted)
  v
SIGNED
  |  delivered over proximity transport
  v
DELIVERED
  |  peer verifies certificates and signature, persists, signs receipt
  v
ACKNOWLEDGED ------------------------------+
  |  settlement worker claims lease        |
  v                                        |
SETTLING                                   |  ambiguity: read back
  |            |                           |  before deciding
  |            +---------------------------+
  |  capture / partial capture
  +--> SETTLED                (terminal, capture id recorded)
  +--> RETRYABLE_FAILURE      (backoff, returns to SETTLING)
  +--> SETTLEMENT_BLOCKED     (terminal until payer re-approves)
  +--> REJECTED_CONFLICT      (terminal, alert raised, vault blocked)
  +--> REJECTED_OVERSPEND     (terminal, alert raised, vault blocked)
```

An exact duplicate is not a state change. The backend returns the stored outcome flagged as a replay and makes no PayPal call.

TRANSITION	GUARD	ON FAILURE
DRAFT to SIGNED	Certificates valid; vault before spend-by; capacity sufficient; sequence persisted	Refuse and state the remaining capacity
SIGNED to DELIVERED	Transport reports delivery or a receipt is received	Remain SIGNED and resend the same bytes
DELIVERED to ACKNOWLEDGED	Certificates and signature verify; vault and sequence unseen on this phone	Refuse with a visible reason; no pending credit
ACKNOWLEDGED to SETTLING	Worker holds an exclusive lease	Leave queued
SETTLING to SETTLED	PayPal reports completion for this voucher	Record identifier, propagate to both devices
SETTLING to RETRYABLE_FAILURE	Timeout, 429, or 5xx after read-back shows no capture	Exponential backoff with jitter
SETTLING to SETTLEMENT_BLOCKED	Authorisation expired, voided or declined	Surface to both parties; never retry silently
SETTLING to REJECTED_CONFLICT	Same vault and sequence with different content	Alert and block the vault
SETTLING to REJECTED_OVERSPEND	Capture would exceed the authorised amount	Reject the excess voucher, alert and block the vault

3.7 Minimum interface surface

Only what the Android app needs. Every mutating client endpoint is authenticated by device token and idempotent. The API contract in `docs/T-Pay-API-Contract.md` is the binding detail for payloads, errors and the activity that calls each endpoint.

METHOD AND PATH	PURPOSE	IDEMPOTENCY
POST /devices/register	Register a public key, return device token, key identifier and device certificate	Idempotent on public key
POST /vaults	Create the PayPal order and return the approval link	One active vault per user per currency
GET /paypal/return	Hosted page that returns the approval to the app through a deep link	Read only
POST /vaults/{id}/authorize	Authorise the approved order and return the vault certificate	Keyed on PayPal order identifier
GET /vaults/current	Read authorised, reserved, settled and remaining capacity	Read only
POST /vouchers	Submit a signed voucher for settlement from either party	Unique on vault and sequence
GET /vouchers	List vouchers for the calling device with payer or payee view	Read only
GET /vouchers/{id}	Read one voucher including capture identifier	Read only
POST /vaults/{id}/melt	Stop signing, accept the signed spend statement, and release or schedule release of the remaining hold	Keyed on vault
GET /wallet	Read the Wallet: collected credits, withdrawals and balance	Read only
POST /wallet/withdrawals	Pay out the Wallet balance through PayPal Payouts	Keyed on `Idempotency-Key`
POST /assistant/interpret	Turn audio or text into a validated, typed proposal	Read only; signs nothing
POST /webhooks/paypal	Receive PayPal events, verified and idempotent	Keyed on event identifier
GET /health	Report liveness and dependency reachability	Read only

3.8 Definition of done for V1

A reviewer with two Android phones and no cooperation from the build team must be able to:

1. Install the app on both phones and see each device's identity and Tier B assurance badge.
2. Lock a vault while online, completing a real PayPal sandbox approval, and see the authorisation reflected back from PayPal.
3. Put both phones in airplane mode and make a payment; the payer sees a pending debit and the payee a pending credit within a second of the exchange.
4. Attempt a payment larger than the remaining capacity and be refused before a signature exists.
5. Present a tampered voucher to the merchant phone and see it refused while offline.
6. Reconnect one phone only and watch the voucher settle without any user action.
7. Confirm both phones show the settled state, each with the same capture identifier, once both are online.
8. See the merchant ledger credited, and withdraw it through sandbox Payouts if FR-41 is complete.
9. Replay a captured voucher at the API and confirm no second capture occurs.
10. Melt the vault and confirm the remaining hold is released at PayPal, not merely in the app.
11. Use voice in a supported language to propose a payment, confirm it, and hear or see the result.

12. Run the test suite and the typecheck from a clean clone and see both pass.

4. PayPal constraints as requirements

PayPal is not a neutral rail. It constrains the product in ways the concept has to absorb, and several of those constraints are load-bearing: the currency model and the authorisation model each change the design, not just the implementation.

4.1 What PayPal will and will not do

PayPal Orders v2 accepts a fixed list of currency codes: AUD, BRL, CAD, CNY, CZK, DKK, EUR, HKD, HUF, ILS, JPY, MYR, MXN, TWD, NZD, NOK, PHP, PLN, GBP, SGD, SEK, CHF, THB and USD. **NGN is not among them**; a request carrying it fails with `CURRENCY_NOT_ALLOWED`. HUF, JPY and TWD take no decimal places; everything else expects two.

The consequence is unavoidable and better stated plainly than discovered late: a naira amount cannot be held or charged by PayPal. V1 keeps NGN as the currency the user sees and reasons about, and settles in a supported currency at a rate snapshotted when the voucher is signed. That rate is part of the signed payload, so a voucher cannot change value between devices or between issue and settlement.

The second useful discovery is that PayPal's authorisation model maps onto the vault almost exactly. An order created with `intent=AUTHORIZE`, then authorised, puts a hold on the payer's funds; that authorisation can be captured **partially and repeatedly** while `final_capture` remains false; and the remainder is released by voiding the authorisation. Lock, spend, melt therefore become create-and-authorise, partial capture, void — with PayPal itself enforcing the ceiling rather than trusting our arithmetic.

The caveats are real: an authorisation's honor period is three days and its validity twenty-nine. A voucher whose authorisation has expired must become a blocked state, not a retry that spins forever. Both phones may stay offline for days, and the customer may open PayPal without opening T-Pay, so a short settle-by would release money that offline merchants are owed. Spend-by stays at most 24 hours after lock. Settle-by is a day before the authorisation expires, at most 28 days after lock. Each time an honor period ends, the backend reauthorises the uncaptured amount, which gives a new authorisation ID with a fresh three-day guarantee.

PayPal states that multiple partial captures must be enabled on the account for PayPal payments. M0 proves partial capture, remaining-hold release and Payouts on the project's sandbox accounts before any dependent code is written.

The third constraint is the payee. The capture request carries amount, invoice identifier, final-capture flag, payment instruction, note and soft descriptor, but no payee. Captures therefore go to the account on the authorised order, which is the T-Pay business sandbox account. The T-Pay ledger tracks what each merchant is owed, and Payouts moves that balance out. Live operation of this model would raise licensing and PayPal programme questions outside this release.

4.2 Requirements

ID	REQUIREMENT	ACCEPTANCE CRITERION	SOURCE
PP-01	Never send NGN to PayPal. All amounts crossing the API use a supported currency code with the correct decimal precision, converted through the FX port with the rate recorded on the voucher.	Requesting any PayPal operation with NGN fails before the call is made; a test asserts the currency is in the allowed set and the value matches the required precision.	developer.paypal.com/reference/currency-codes
PP-02	Client identifier and secret exist only in backend environment variables. Clients receive only an identifier where the SDK requires one, never a secret.	Bundle search finds no secret; rotating the secret requires no client release.	developer.paypal.com/api/rest/authentication/
PP-03	The OAuth token is cached in memory and refreshed before `expires_in` elapses; one token per settlement is a defect.	A batch of ten settlements produces one token request.	developer.paypal.com/api/rest/authentication/
PP-04	The vault is an authorisation: create with `intent=AUTHORIZE`, authorise, then capture partially with `final_capture=false`, recording the voucher identifier as `invoice_id`.	Each settled voucher corresponds to one capture against the vault's authorisation identifier; the sum of captures never exceeds the authorised amount.	developer.paypal.com/v5/checkout/auth-capture

ID	REQUIREMENT	ACCEPTANCE CRITERION	SOURCE
PP-05	Authorisation lifetime is respected: settle-by falls a day before the 29-day authorisation expires, each honor period is renewed by reauthorisation, and expiry is a blocking state.	A simulated expired authorisation produces SETTLEMENT_BLOCKED, not a retry loop; no vault has settle-by beyond 28 days after lock; a vault nobody melts is reauthorised on day three and a day-one voucher settles on day eight.	developer.paypal.com/docs/checkout/standard/customize/authorization-honor
PP-06	Own the deduplication. The database is the authority keyed on voucher identity; `PayPal-Request-Id` is a UUID bound to the payload hash for a single logical attempt. Assume PayPal remembers it for six hours at most.	A duplicate submitted eight hours later is still deduplicated by our store; the same key is never reused with a different body.	developer.paypal.com/api/rest/reference/idempotency
PP-07	Reconcile before retry. On timeout or 5xx, read the order or authorisation back before deciding; retry only timeouts and 5xx, with backoff and jitter.	A forced timeout with a completed capture results in one capture and a settled voucher.	developer.paypal.com/api/make-api-requests
PP-08	Webhooks are the source of truth; polling is a bounded reconciliation fallback. Handlers verify signatures, are idempotent, and accept both decline spellings.	Replaying an event is a no-op; both `PAYMENT.CAPTURE.DECLINED` and `PAYMENT.CAPTURE.DENIED` are handled.	developer.paypal.com/api/rest/webhooks/event-names

ID	REQUIREMENT	ACCEPTANCE CRITERION	SOURCE
PP-09	Respect rate limiting: no polling loops, serialised writes per voucher, backoff on 429.	A 429 in a stubbed gateway delays the next attempt by the configured backoff.	developer.paypal.com/api/rest/reference/rate-limiting
PP-10	Sandbox only, against the sandbox base URL, with an explicit refusal to run live mode.	Live configuration is rejected at startup with a clear message.	developer.paypal.com/api/rest/integration/orders-api
PP-11	Handled events are exactly: capture completed, pending, declined, denied, refunded, reversed; authorisation voided; and payment-approval reversed.	Each event maps to a documented voucher transition or is explicitly ignored with a log line.	developer.paypal.com/api/rest/webhooks/event-names
PP-12	Settlement failures are classified into retryable, blocked and rejected classes, because conflating them is what produces both infinite retries and silently lost money.	Each class has a test asserting the resulting voucher state and the operator-visible message.	developer.paypal.com/api/rest/integration/payments-api/v1-v2-migration
PP-13	Captures against one authorisation are serialised, because concurrent partial captures on one hold race for the same capacity.	Concurrent vouchers on one vault produce ordered captures, and the database row lock is held across capacity checks.	Project design

ID	REQUIREMENT	ACCEPTANCE CRITERION	SOURCE
PP-14	The authorised order names only the T-Pay business sandbox account as payee. No code attempts to redirect a capture to a merchant.	Capture payloads contain no payee; each merchant's share exists in the T-Pay ledger.	raw.githubusercontent.com/paypal/paypal-rest-api-specifications/main/openapi/payments_payment_v2.json
PP-15	Merchant withdrawals use the Payouts API with a unique sender batch identifier.	A retried withdrawal produces one payout.	developer.paypal.com/docs/payouts
PP-16	M0 runs a sandbox proof covering authorise, two partial captures, remaining-hold release, a webhook and one payout.	A script records each PayPal response. Any failure updates this document before dependent work begins.	Project design

4.3 Consequences the product must accept

- **The vault holds a supported currency, not naira.** Displaying ₦10,000 against a USD hold requires a rate; that rate is signed into every voucher, and a rate change between lock and spend is visible rather than silent.
- **Authorisation is not a debit.** Money is held, not taken. The melt operation is a void, and calling it a refund would misdescribe the system.
- **Hold validity bounds offline life.** A voucher older than the authorisation's remaining validity cannot settle. The product therefore needs a stated maximum offline window, enforced at signing time.
- **The demo settles in USD.** The sandbox proof in PP-16 confirms it. Naira remains the display currency.
- **T-Pay sits in the money flow.** Because captures reach the T-Pay account first, the merchant's balance depends on the T-Pay ledger. The UI must say so.
- **Offline acceptance carries residual risk.** A merchant accepts a voucher before PayPal has captured it. Certificates and caps reduce the risk; settlement detects the remainder.

4.4 Sources

TOPIC	URL
Currency codes	https://developer.paypal.com/reference/currency-codes
Currency error	https://developer.paypal.com/api/errors/currency-not-allowed
Idempotency	https://developer.paypal.com/api/rest/reference/idempotency

TOPIC	URL
Making API requests	https://developer.paypal.com/api/make-api-requests
Orders API integration	https://developer.paypal.com/api/rest/integration/orders-api
Authorise then capture	https://developer.paypal.com/v5/checkout/auth-capture
Reauthorise	https://developer.paypal.com/api/payments/v2/authorizations-reauthorize
Capture an authorisation	https://developer.paypal.com/api/payments/v2/authorizations-capture
Capture request schema	https://raw.githubusercontent.com/paypal/paypal-rest-api-specifications/main/openapi/payments_payment_v2.json
Authorization and honor period	https://developer.paypal.com/docs/checkout/standard/customize/authorization-honor/
Payouts	https://developer.paypal.com/docs/payouts/
Webhook event names	https://developer.paypal.com/api/rest/webhooks/event-names
Authentication	https://developer.paypal.com/api/rest/authentication/
Rate limiting	https://developer.paypal.com/api/rest/reference/rate-limiting
v1 to v2 migration	https://developer.paypal.com/api/rest/integration/payments-api/v1-v2-migration
Orders v2 OpenAPI specification	https://raw.githubusercontent.com/paypal/paypal-rest-api-specifications/main/openapi/checkout_orders_v2.json

5. Device trust

The concept calls for a secure enclave on both devices. The hackathon build uses Tier B on Android, and saying so is part of the requirement.

5.1 Two assurance tiers

TIER	PLATFORMS	WHAT IT GIVES	WHAT IT DOES NOT GIVE
A — hardware-backed	Android Keystore with StrongBox	Private key never enters application memory; signing is a hardware operation; optional attestation	Nothing about the honesty of the amount being signed
B — software-protected	Android with Expo: software key encrypted at rest through `expo-secure-store`	A copy of application storage alone is not enough to sign	The key enters application memory while signing, and an attacker controlling the device can sign

Tier B is acceptable for this release and must be labelled as such wherever assurance is claimed. The release build goes one step further without changing the label: a local native module creates the key inside the Android Keystore, in StrongBox when the phone has it, so the private key never enters application memory and signing is a Keystore operation. It stays Tier B because the backend does not yet verify key attestation, so it can't prove to anyone else where the key lives. Web and development builds fall back to a `@noble/curves` key in `expo-secure-store`. Randomness comes from the platform through `expo-crypto`. Reaching Tier A means adding attestation checks at registration, and stays parked until after submission.

5.2 Signing rules

ID	RULE
DT-01	ECDSA over P-256 with SHA-256 is the single algorithm. The app signs with `@noble/curves`, and the backend verifies with Node cryptography.
DT-02	Payloads are canonicalised with JCS (RFC 8785). Never sign `JSON.stringify` output: key order, number formatting and unrelated whitespace differ between JavaScript engines.
DT-03	Signatures are transmitted as base64url of the raw 64-byte `r‖s` pair and normalised at the boundary. Node verification uses the `ieee-p1363` encoding.
DT-04	A domain-separation prefix is prepended to the canonical bytes, so a signature over a voucher can never be replayed as a signature over something else.
DT-05	Amounts are integer minor units or decimal strings. Floating-point values never enter a signed payload.
DT-06	The payload binds key identifier, device identifier, vault identifier, sequence, NGN amount, settlement amount, FX rate, payee key, payment request nonce, issue time and expiry.
DT-07	Replay defence is layered: a monotonic sequence persisted before issue; a merchant-side seen list; a request nonce; a server unique key on vault and sequence; and an expiry. No single layer is trusted alone.
DT-08	Double-spend detection is online reconciliation: a voucher that conflicts with a stored sequence, or would exceed the authorisation, is rejected, blocks that vault and raises an alert.
DT-09	Trust is rooted in a T-Pay signing key held only by the backend. Its public key is pinned in the app. Device and vault certificates use the DT-02 to DT-04 rules with their own domain-separation prefixes. Certificates live no longer than 7 days for devices and the vault's settle-by time for vaults.
DT-10	Sequence order carries no meaning at the backend. The identity of a voucher is its vault and sequence, and the integrity check is its payload hash.

5.3 Why the layering matters

The Bank of England's 2025 digital pound offline-payments experiment reached the same conclusion from the other direction: vendors relied on secure elements for key protection, and all four solutions used online reconciliation as the fallback that detects counterfeiting and double spending. The report is explicit that a breached secure element would permit both. Our design follows that finding rather than assuming hardware alone is sufficient.

5.4 Sources

TOPIC	URL
JCS, RFC 8785	https://www.rfc-editor.org/rfc/rfc8785.html
Android Keystore	https://developer.android.com/privacy-and-security/keystore
Expo SecureStore	https://docs.expo.dev/versions/latest/sdk/securestore/
react-native-keychain	https://oblador.github.io/react-native-keychain/docs/platform-value-storage/
Offline payments experiment	https://www.bankofengland.co.uk/report/2025/digital-pound-experiment-report-offline-payments
GNU Taler double-spend detection	https://www.taler.net/en/news/2025-01.html

6. Architecture contract

These are constraints, not preferences. Each rule names the specific place in the current codebase that violates it, because a rule that has never been broken is a rule nobody needs.

6.1 Rules

ID	RULE	RATIONALE	CURRENT VIOLATION
AR-01	Dependencies point inward only: domain depends on nothing outside the shared kernel; application depends on domain and ports; infrastructure depends on application and domain; presentation depends on application.	A domain that imports a framework cannot be tested, reused or reasoned about.	None at present; the violation arrives if the new storage port is skipped in W-19.
AR-02	No platform global below the adapter layer. `window`, `navigator`, `document`, `fetch`, `localStorage`, `fs` and React Native modules may appear only inside an infrastructure adapter.	The same domain must run in React Native and in Node.	`PayPalSdkGatewayAdapter` reads `navigator.onLine` and `localStorage`; every `LocalStorage*Repository` touches `window`.
AR-03	Single ownership of state. A domain fact has exactly one owner; a view reads it and never mirrors it.	Two copies of a balance is two answers to the same question.	`PayPalEscrowModal` keeps its own vault amount, spent amount and token list beside the domain.
AR-04	One implementation per behaviour.	Duplicated logic diverges silently.	`VoiceAssistantModal` contains an inline intent parser beside `LocalSLMAdapter`.
AR-05	Every external edge is a port: network, storage, cryptography, clock, randomness, speech, proximity. Aggregates never read the wall clock or generate randomness directly.	Non-deterministic aggregates cannot be tested and cannot be replayed.	`MarketVault.create` and `CryptographicExchangeService` call `new Date()` and `Math.random()` inline.
AR-06	Only the composition root constructs adapters, and nothing reaches a singleton from inside the domain.	Wiring in one place is what makes substitution possible.	Hooks and components reach the global container directly.
AR-07	No speculative code. Unused exports, entities with no use case and flags for the future are deleted rather than kept.	Unused code is untested code that still has to be maintained.	`CryptographicExchangeService.verifyProof` is never called; parts of the escrow event set are never published.
AR-08	Claims must be executable. A capability may be described as working only if a test or a reachable UI path exercises it.	The gap between README and behaviour is itself a defect.	`README.md` asserts signatures, mesh networking, a secure enclave and a working PayPal integration.
AR-09	Every mutating API is authenticated by device token and idempotent.	A retried request must not be a second payment.	The REST layer accepts any caller as any user.

6.2 Bounded context ownership

CONTEXT	OWNS	MAY IMPORT	MUST NEVER TOUCH
Shared kernel	Identity, money, result, aggregate base, event bus	Nothing	Any context
Escrow	Vault, voucher, cryptogram, settlement state	Shared kernel	Tracker or assistant internals

CONTEXT	OWNS	MAY IMPORT	MUST NEVER TOUCH
Merchant	Inventory, sale, price derivation, reconciliation	Shared kernel	Vault or settlement internals
Budget	Plan, categories, expenses, ambiguity	Shared kernel	Vault or settlement internals
Transit	Trip, seats, fares	Shared kernel, proximity port only	Vault or settlement internals
Assistant	Intent, language, speech	Shared kernel and application use cases	Any aggregate internals directly

Cross-context communication is domain events only. No context may read another's aggregates.

6.3 Import matrix

FROM → TO	SHARED	DOMAIN	APPLICATION	INFRASTRUCTURE	PRESENTATION
Shared	Yes	No	No	No	No
Domain	Yes	Same context	No	No	No
Application	Yes	Yes	Same context	Ports only	No
Infrastructure	Yes	Yes	Yes	Same platform	No
Presentation	Types only	No	Yes	No	Same layer

6.4 Review checklist

Run before any change is accepted, in this order. A "yes" answer needs a justification in the pull request.

1. Does it add an import across a boundary in 6.3? (AR-01)
2. Does it introduce a platform global outside an adapter? (AR-02)
3. Does it create a second holder of a fact that already has an owner? (AR-03)
4. Does it duplicate behaviour that exists elsewhere? (AR-04)
5. Does it read the clock or randomness outside a port? (AR-05)
6. Does it construct an adapter outside the composition root? (AR-06)
7. Does it add an export nothing calls, a flag nothing reads, or a type nothing uses? (AR-07)
8. Does it claim a capability that no test or reachable path exercises? (AR-08)
9. Does it add a mutating endpoint without authentication and idempotency? (AR-09)

6.5 Enforcement

Rules that rely on memory are not rules. The first two gates start in M0, and the third starts in M9 because deleting the web prototype's exports would otherwise dominate the deadline.

GATE	MECHANISM	CATCHES
Import boundaries	Static rule file checked in CI, expressing the matrix in 6.3	AR-01, AR-05, AR-06
Forbidden globals	Grep gate over everything outside `infrastructure/` adapters	AR-02
Unused exports	Dead-code check on the domain and application layers	AR-07

6.6 Project structure

The repository root has three project folders. The offline payment cannot call a server, so the phone must be able to sign and verify vouchers itself. The backend must apply the same rules when settling, so those rules live once in `shared/` (AR-04).

```
T-Pay/
  frontend/                  Everything the user sees
    App.tsx, app.json        Expo Android app, rebuilt in M2
    web-prototype/           Retired web demo, kept as design reference
  backend/                   Server functions on Render
    src/controllers/         HTTP handlers
    src/routes/              HTTP API
    src/services/            settlement, certificates, ledger (M1 onwards)
    src/adapters/            PayPal, Postgres, Gemini (M1 onwards)
  shared/                    Payment rules used by both
    src/escrow/              vault, voucher, device identity
    src/shared/              money, result, aggregate base
    src/resource-tracking/   merchant, budget, transit
    src/hybrid-assistant/    intent and language rules
  docs/                      PRD, generated documents and their build scripts
```

FOLDER	HOLDS	MUST NEVER HOLD
`frontend/`	Screens, device storage, keys, proximity transport and speech	PayPal or Gemini secrets, or settlement decisions
`backend/`	API, PayPal, Postgres, certificates, settlement, ledger, Gemini	Device private keys or interface code
`shared/`	Pure TypeScript domain and application rules	Expo, Express, database, network or secret code

Each project folder installs its own dependencies, so the repository root holds no `package.json` or `node_modules`. The root keeps only project-wide files such as `README.md`, `.gitignore` and `LICENSE`. The backend and web prototype link `@tpay/shared` as a local package, and the Expo app does the same in M2. The old paths moved as follows: `react-native-app/` became `frontend/`; web `src/` became `frontend/web-prototype/`; `core/` became `shared/src/`. Evidence paths in section 1 and Appendix A refer to the audited commit `8da2e8d`. Browser adapters and the shared composition root are still in `shared/src/` and move out or are replaced during M1.

7. Roadmap

Ordered by dependency, not by enthusiasm, with target dates for the 12 November 2026 deadline. Each milestone states the outcome a reviewer can observe, not the work performed.

ID	MILESTONE	OUTCOME	EXIT CRITERIA	CLOSES
M0	Baseline and sandbox proof (4 Nov)	The repository can run, and PayPal assumptions are proven	`LICENSE` added; backend starts; typecheck and test runner pass; `strict` on; `Money` fixed; attack tests committed and failing; PP-16 complete	W-01, W-20, G-N06
M1	Shared core and backend (5 Nov)	One domain, two runtimes, one deployed backend	Repository restructured as in 6.6; `shared/` runs in Node and React Native; clock and randomness ports exist; Postgres and Render deployment work; device authentication works	W-02, W-19, G-N07
M2	Android app and identity (5 Nov)	Each phone can prove who it is	Expo SDK 58 app builds outside Expo Go; keypair persisted; device certificate issued; Tier B badge displayed	W-08, W-18, G-F01
M3	Signed vouchers and two-phone exchange (6–7 Nov)	Two airplane-mode phones exchange an unforgeable voucher	JCS fixtures pass; QR exchange and receipt work; merchant seen list works; Nearby test passes within half a day or is dropped	W-09, W-10, W-15, W-25, G-F02, G-F03, G-F13, G-F14
M4	Vault as authorisation (7 Nov)	PayPal enforces the ceiling	App approval creates a real authorisation; vault certificate issued; signing respects capacity; melt follows FR-09	W-06, W-11, W-12, W-14, G-F04, G-F05, G-F08, G-F15
M5	Server-side settlement (8 Nov)	Money moves in one place, once	Idempotent serialised capture; webhooks handled; retry classification; capture identifiers recorded; duplicate and conflict tests pass	W-13, G-F06, G-F07, G-N03, G-N04
M6	Dual-device ledger and sync (9 Nov)	Both phones agree	Reconnect-triggered settlement; outcome propagated; merchant ledger credited; reconciliation flags overspend	W-16, W-26, G-F09, G-F10, G-F16
M7	Real AI assistant (9 Nov)	Voice works, honestly	Model proposal validated and confirmed; rule-based offline fallback labelled; spoken output with fallback	W-21, G-F11
M8	Complete product experience (10 Nov)	The app feels finished	Fake flows removed; five-language payment strings; empty, error and blocked states designed; inventory connected if time allows	W-23, G-N08, G-N09
M9	Durability and verification (10 Nov)	Failures are caught before judges find them	Kill test; concurrency test; attack tests green; dead-code gate; README matches reality	W-22, G-N01, G-N05
M10	Submission (11–12 Nov)	A judge can verify the claims unaided	Section 3.8 executed on two phones; signed APK released; video published; Devpost submission complete before the deadline	W-24, G-N10, All

Sequencing is strict in one place: M2 before M3, M3 before M4, M4 before M5, M5 before M6. Everything else may overlap. If the schedule slips, cut M8 inventory, then FR-41 Payouts, then Nearby. Never cut a P0 security or settlement requirement.

7.1 Parked

The list in 1.11 is normative for the whole roadmap. No parked item may be started, scoped or prototyped before M10 is signed off.

8. Open inputs and decisions

ID	OPEN ITEM	WHY IT MATTERS	OWNER DECISION NEEDED
O-01	Page 5 of the source PDF	Reviewed: it compares B2B and closed-loop strategy	Closed; no requirement added
O-02	Whether the sandbox supports partial capture, hold release and Payouts as designed	The vault and merchant funds path depend on it	Run PP-16 first; requires a PayPal developer account
O-03	Which assurance tier the demo presents	Presenting Tier B as hardware would be false	Decided: Tier B, labelled
O-04	Source of the FX rate	A signed rate implies an audit trail	Proposed: fixed demo rate labelled as such; confirm the number
O-05	Maximum offline window	Bounded by PayPal's 29-day authorisation and product intent	Decided 10 November: spend-by at most 24 hours; settle-by a day before the authorisation expires, at most 28 days after lock
O-06	The web build's role	The product is an app	Decided: retired as a product surface; remove it after the mobile screens replace it
O-07	Who absorbs a rejected overspend after launch	Merchants need to know whether an offline pending credit is guaranteed	Proposed for the demo: not guaranteed, with a per-voucher cap; decide a post-hackathon policy
O-08	Whether the team has two Android phones and one person for each demo role	The video must show the app on the device it was built for	Confirm by 5 November
O-09	Whether the entrant owns the code and may publish under MIT in the ATA organisation repository	The rules require original, entrant-owned, open source work	Confirm with the repository owner
O-10	Native-speaker review for Yoruba, Igbo, Hausa and Pidgin	Machine-written demo lines could be wrong, and device voices may be missing	Name a reviewer for each language

9. Hackathon delivery

The controlling document is the official rules page. If it changes, this section changes.

9.1 Rules that shape the build

RULE	CONSEQUENCE
Submission closes 12 November 2026 at 12:00 Pacific Time	That is 21:00 in Nigeria. The internal target is 11 November.
Project must integrate PayPal sandbox and an AI tool, and PayPal must be central	Both are P0. A keyword parser would not satisfy the AI requirement.
Project must be new or significantly updated in the submission period	The first repository commit is dated 27 October 2026, inside the period.
Judges need a working build	Publish a signed APK, full setup steps, and sandbox test logins. A mockup fails.
Public repository with a detectable open source license	Add `LICENSE` to the root and confirm GitHub detects it.
Video under three minutes, on YouTube, showing the app on its device	Film two real phones. Avoid unlicensed music and third-party brands, including drink and fintech names.
Submission materials in English	Add English captions to any other language in the video.
Judging is equally weighted across implementation, design, impact, innovation and presentation	Finish the product experience; do not stop at a working protocol.

9.2 Submission checklist

ITEM	OWNER EVIDENCE
Text description	Names the problem, the audience, the PayPal products used, the AI model used and how it is used
Functional access	Signed APK link, hosted backend, README setup steps and sandbox test logins that stay valid until judging ends
Public code	All source and assets, with no secrets committed
License	MIT `LICENSE` detected in the repository About section
Video	Under three minutes, public on YouTube, following the demo script below
Honesty	Tier B, the fixed demo rate, rule-based offline voice and sandbox use are stated plainly

9.3 Stack

LAYER	CHOICE	REASON
Packages	Separate npm packages in `frontend/` and `backend/`, each linking the local `shared/` package	Keeps each side self-contained with one source for the payment rules
Mobile app	Expo SDK 58, React Native, TypeScript, EAS or local Android build	The product is an Android app, and Nearby requires a native build
Keys and signing	`@noble/curves`, `expo-crypto`, `expo-secure-store`	Provides P-256 signing in React Native with the protection stated in 5.1
Canonical bytes	`canonicalize`	Implements JCS for DT-02

LAYER	CHOICE	REASON
Local data	`expo-sqlite`	Durable voucher, sequence and ledger storage
Device exchange	`expo-camera`, `react-native-qrcode-svg`, optional `expo-nearby-connections`	QR is the guaranteed path; Nearby is the upgrade
PayPal approval	`expo-web-browser`, deep links	Opens real approval and returns to the app
Voice	`expo-audio`, `expo-speech`, `expo-speech-recognition`	Records audio, speaks results, and uses offline recognition where the device supports it
Backend	Node, TypeScript, Express on Render	Holds secrets, certificates and settlement
Database	Render Postgres	Unique keys and row locks enforce settlement safety
PayPal	Orders v2, Payments v2, Webhooks and Payouts	Covers hold, capture, release, event handling and merchant withdrawal
AI	Gemini through `@google/genai` on the backend	The dependency already exists and the model can take audio and return structured output
Testing	Vitest, fast-check, Postman	Domain, property and API testing

No sponsor tool is required. Render and Postman are used because the project needs them. APIMatic's PayPal Context Plugin may be used during development. AG Studio and Render Workflows are stretch work after M10. The other sponsor tools are out of scope.

9.4 Video script

1. Show the problem: a market sale stops when the network and POS fail.
2. Lock a vault with a real PayPal sandbox approval.
3. Turn on airplane mode on both phones.
4. Ask for a payment by voice in Pidgin, confirm it, and exchange it between the phones.
5. Show pending debit and pending credit, then a refused overspend and a refused tampered voucher.
6. Reconnect one phone and show automatic settlement.
7. Show the same capture identifier on both phones and the capture in the PayPal sandbox.
8. Melt the vault, show the hold released, and close on the impact.

9.5 Prize targets

The project may win one grand prize and one sponsor prize, or one honorable mention and one sponsor prize. The natural targets are the grand prizes, Most Impactful, Best Use of PayPal + AI and Best Demo Delivery, with Best Use of Render as the sponsor target.

9.6 Sources

TOPIC	URL
Official rules	https://paypalaihackathon.devpost.com/rules
Render resources	https://paypalaihackathon.devpost.com/details/render
AG Grid resources	https://paypalaihackathon.devpost.com/details/aggrid

TOPIC	URL
APIMatic resources	https://paypalaihackathon.devpost.com/details/apimatic
Postman resources	https://paypalaihackathon.devpost.com/details/postman

Appendix A. Evidence index

PATH	SUPPORTS
`core/shared/domain/`	W-01
`core/escrow/domain/`	W-02, W-06, W-11
`core/escrow/application/ports/`	W-02
`core/escrow/domain/services/CryptographicExchangeService.ts`	W-09
`core/escrow/domain/value-objects/DeviceIdentity.ts`	W-08
`core/escrow/infrastructure/adapters/PayPalSdkGatewayAdapter.ts`	W-13, W-14
`core/escrow/infrastructure/adapters/BroadcastChannelProximityAdapter.ts`	W-15
`core/escrow/infrastructure/repositories/LocalStorageEscrowRepository.ts`	W-19
`core/resource-tracking/merchant/`	W-03
`core/resource-tracking/budget/`	W-04
`core/resource-tracking/transit/`	W-05
`core/hybrid-assistant/infrastructure/adapters/LocalSLMAdapter.ts`	W-21
`core/container.ts`	W-07, W-15
`core/__tests__/domain-verification.ts`	W-07, W-11
`src/components/PayPalEscrowModal.tsx`	W-17
`src/components/VoiceAssistantModal.tsx`	W-17, W-21
`src/components/views/TrackerView.tsx`	W-17
`src/hooks/useTPayDomain.ts`	W-16
`react-native-app/App.tsx`	W-18
`backend/src/server.ts`	W-20
`backend/src/controllers/vault.controller.ts`	W-20
`README.md`	W-23
`DDD_ARCHITECTURE.md`	W-23

Appendix B. Trace from the source document

SOURCE MATERIAL	PRODUCES	STATUS TODAY
Hybrid assistant that works online and offline (p1)	FR-31, FR-32	Simulated
Lightweight on-device model, any technology (p1)	FR-31; on-device model parked	Simulated
Resource tracker tracking stock, budget or seats by use case (p1)	FR-26 to FR-29	Met in domain, Partial in UI
Three demo cases: trader, salaried user, bus driver (p1)	FR-26 to FR-29	Met in domain
Spoken sale with price-to-quantity derivation (p1)	FR-26	Met
Budget deduction with category and clarification (p2)	FR-28	Met in domain
Seven-seat korope with collect-from-everybody (p2)	FR-29	Met in domain
Spoken inventory setup with read-back in a chosen language (p2)	FR-27, FR-33	Met in domain, Simulated in UI
End-of-day stock review and reminder (p2)	FR-30	Missing
Both devices exchange a token identifying each other (p3)	FR-01, FR-02, FR-11, FR-22, FR-36, FR-38	Missing
Whichever device reconnects first reports to PayPal (p3)	FR-23, FR-24	Simulated
Offline-first default, settle silently when online (p3)	FR-15	Partial
Funds locked in an offline wallet for a period and returned if unused (p3)	FR-06, FR-09, restated as a PayPal hold rather than a wallet	Simulated
PayPal as primary processor, SDK and API requirements (p1, p3)	PP-01 to PP-12	Simulated
Offline protocol as an API for banks (p4, p6, p7)	Parked, see 1.11	Not in V1